

SQL Master

Complete Syntax, Patterns & Interview Guide

 Full Syntax

 Common Mistakes

 Pattern Recognition

 Practice Links

 50+ Interview Q&A

How to use this doc: Every concept follows the same 4-part formula so you never get lost:

1.  **Syntax** — the exact way to write it (with a colored code block).
2.  **Common Mistake** — the syntax trap where people lose marks (like `WHERE` with a window function).
3.  **Pattern Recognition** — the keywords in a question that scream "use this concept."
4.  **Practice** — LeetCode/DataLemur problems to build muscle memory.

At the very end there's a **50+ most-asked interview questions cheat sheet** for last-minute revision.

The #1 mindset: Interviewers don't ask "explain `ROW_NUMBER`" — they ask "find the 2nd highest salary per department." Your job is **pattern recognition**: hear the question → instantly know which SQL construct fits. That's what this doc trains.

Contents

- | | |
|--|------------------------------------|
| 1. SQL Execution Order (★ MUST know first) | 4. GROUP BY, HAVING & Aggregations |
| 2. SELECT / WHERE / ORDER BY / LIMIT | 5. Joins (all types + self join) |
| 3. NULL Handling (traps galore) | 6. Anti-Join & Semi-Join Patterns |
| | 7. Subqueries (correlated vs not) |

- 8. CTEs — Common Table Expressions
- 9. Recursive CTEs (hierarchies)
- 10. Window Functions ★★ ★ (MOST asked)
- 11. Ranking: ROW_NUMBER vs RANK vs DENSE_RANK
- 12. LEAD, LAG & Consecutive Rows
- 13. Running Totals & Moving Averages
- 14. Gaps & Islands (Hard)
- 15. CASE WHEN & Conditional Logic
- 16. UNION, INTERSECT, EXCEPT
- 17. Pivot & Unpivot
- 18. String Functions
- 19. Date/Time Functions (DE-critical)
- 20. Finding & Removing Duplicates
- 21. Top-N per Group Pattern
- 22. DDL / DML / DCL / TCL
- 23. Indexes (Clustered vs Non-clustered)
- 24. Views & Materialized Views
- 25. Stored Procedures & Functions
- 26. Triggers
- 27. ACID, Transactions & Isolation
- 28. Normalization (1NF, 2NF, 3NF, BCNF)
- 29. Star vs Snowflake Schema (DE)
- 30. Query Optimization & EXPLAIN
- 31. 🎯 Complete Pattern Recognition Cheat Sheet
- 32. 🧠 50+ Most-Asked Interview Q&A

1. SQL Execution Order (★ Know This Before Anything Else)

You **write** SQL in one order, but the engine **runs** it in a completely different order. Knowing this explains 90% of "why can't I use X here?" errors — especially the classic "**can't filter with a window function's alias in WHERE.**"

- | | | |
|---|-----------------|---|
| 1 | FROM | Pick the source table(s) — this runs first, not SELECT! |
| 2 | JOIN | Combine tables via joins |
| 3 | WHERE | Filter individual rows (before grouping) |
| 4 | GROUP BY | Collapse rows into groups |
| 5 | HAVING | Filter groups (after grouping) |

- 6 **SELECT** Pick columns — window functions calculated here
- 7 **DISTINCT** Remove duplicates
- 8 **ORDER BY** Sort — can use SELECT aliases
- 9 **LIMIT** Take top N rows (last step)

★ KEY INTERVIEW POINT

Why this matters (interviewers love this):

- You **can't use a SELECT column alias in WHERE** — WHERE runs before SELECT. Use a CTE/subquery.
- You **can't use a window function in WHERE** — window functions calculate in SELECT, after WHERE. Wrap in a CTE.
- You **can use aliases in ORDER BY** — ORDER BY runs after SELECT.
- Use **WHERE** to filter **rows before aggregation**; use **HAVING** to filter **groups after aggregation**.

INTERVIEW Q

"Why can't I write `WHERE row_num = 1` when `row_num` is from `ROW_NUMBER()`?"

Because WHERE runs at step 3, but window functions calculate at step 6 (SELECT). The alias doesn't exist yet. Wrap the window function in a CTE, then filter in the outer query.

```
-- ❌ WRONG: doesn't work
SELECT name, ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn
FROM employees
WHERE rn = 1; -- error: rn doesn't exist yet

-- ✅ RIGHT: wrap in a CTE
WITH ranked AS (
  SELECT name, ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn
  FROM employees
)
SELECT name FROM ranked WHERE rn = 1;
```

2. SELECT / WHERE / ORDER BY / LIMIT (Foundation)

SYNTAX

```
SELECT DISTINCT col1, col2, ...  
FROM table_name  
WHERE condition  
ORDER BY col1 [ASC|DESC], col2 [ASC|DESC]  
LIMIT n [OFFSET m];
```

WHERE OPERATORS YOU'LL ACTUALLY USE

- **Comparison:** `=`, `<>` (or `!=`), `<`, `>`, `<=`, `>=`
- **Range:** `BETWEEN a AND b` (inclusive both sides)
- **List:** `IN (v1, v2, ...)`, `NOT IN (...)`
- **Pattern:** `LIKE 'A%'` (starts with A), `LIKE '%son'` (ends), `LIKE '_o%'` (single-char wildcard)
- **NULL:** `IS NULL`, `IS NOT NULL` (never `= NULL`!)
- **Logical:** `AND`, `OR`, `NOT`

✗ COMMON MISTAKE

Can't use SELECT alias in WHERE!

`SELECT salary * 12 AS annual FROM emp WHERE annual > 500000` → **ERROR**. WHERE runs before SELECT. Fix: repeat the expression, or wrap in a CTE, or use the alias in HAVING/ORDER BY (which run after SELECT).

✗ COMMON MISTAKE

`NOT IN` with NULLs silently returns nothing!

If the subquery in `NOT IN (subquery)` can return NULL, the whole thing gives 0 rows. Always add `WHERE col IS NOT NULL` in the subquery, or use `NOT EXISTS` instead — it's NULL-safe.

LIMIT VS TOP VS FETCH (DIALECT DIFFERENCES)

Database	Syntax
MySQL, PostgreSQL, SQLite	<code>LIMIT 10 OFFSET 20</code>
SQL Server	<code>SELECT TOP 10</code> or <code>OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY</code>
Oracle (12c+)	<code>FETCH FIRST 10 ROWS ONLY</code>

PATTERN RECOGNITION

Simple filter or pull top N? → basic SELECT + WHERE + ORDER BY + LIMIT. Don't over-engineer.

PRACTICE — BASICS (EASY)

1. **Easy** LC 1757. Recyclable and Low Fat Products
2. **Easy** LC 584. Find Customer Referee
3. **Easy** LC 595. Big Countries
4. **Easy** LC 1148. Article Views I
5. **Easy** LC 1683. Invalid Tweets
6. **Easy** LC 175. Combine Two Tables (uses LEFT JOIN — preview)

 [LeetCode SQL 50 Study Plan](#) · [DataLemur \(real interview qs\)](#) · [HackerRank SQL](#)

3. NULL Handling — The Silent Killer

NULLs cause more subtle bugs than any other SQL feature. Master this to look senior.

THE 5 NULL RULES EVERYONE MUST KNOW

1. `NULL = NULL` is **NOT true** — it's NULL (unknown). Use `IS NULL` / `IS NOT NULL`.
2. **Any arithmetic with NULL → NULL**. `10 + NULL = NULL`, `NULL * 0 = NULL`.
3. **Aggregations ignore NULLs** — except `COUNT(*)` which counts all rows.
4. `NOT IN (... , NULL, ...)` **returns no rows**. Avoid; use `NOT EXISTS`.
5. **NULLs in GROUP BY form their own group** (all NULLs group together).

ESSENTIAL NULL FUNCTIONS

```
-- Replace NULL with a default
COALESCE(col1, col2, 'default') -- returns first non-NULL
IFNULL(col, 0)                  -- MySQL
ISNULL(col, 0)                  -- SQL Server
NVL(col, 0)                     -- Oracle

-- Return NULL if two things are equal
NULLIF(a, b) -- returns NULL if a=b, else a. Great for avoiding /0.

-- Count NULLs
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) FROM t;
```

✗ COMMON MISTAKE

`COUNT(col)` vs `COUNT(*)`: `COUNT(*)` counts **all rows**; `COUNT(col)` counts only **non-NULL** values in that column. Interviewers ask this constantly.

✗ COMMON MISTAKE

Avg wrong because of NULLs: `AVG(bonus)` ignores NULLs — so the denominator is smaller than expected. If you want NULLs to count as 0, use `AVG(COALESCE(bonus, 0))`.

🎯 PATTERN RECOGNITION

Question says "including missing/null/no-match cases" → LEFT JOIN + COALESCE, or `IS NULL` handling in WHERE. Never write `= NULL`.

PRACTICE — NULL HANDLING

1. **Easy** LC 1683. Invalid Tweets
2. **Easy** LC 620. Not Boring Movies
3. **Med** LC 1075. Project Employees I
4. **Med** LC 1174. Immediate Food Delivery II
5. **Med** LC 1907. Count Salary Categories

🔗 [LeetCode Database Problems](#)

4. GROUP BY, HAVING & Aggregations

SYNTAX

```
SELECT group_col, agg_func(col)
FROM table
WHERE row_filter          -- filter BEFORE grouping
GROUP BY group_col
HAVING group_filter      -- filter AFTER grouping
ORDER BY ...;
```

AGGREGATE FUNCTIONS

COUNT(), SUM(), AVG(), MIN(), MAX(), STDDEV(), VARIANCE(),
GROUP_CONCAT() / STRING_AGG().

THE WHERE VS HAVING RULE (★ MOST-ASKED)

WHERE

Filters **individual rows** *before* grouping.

Cannot use aggregate functions. `WHERE`

`COUNT(*) > 5` is **illegal**.

HAVING

Filters **groups** *after* GROUP BY runs.

Can use aggregate functions. `HAVING`

`COUNT(*) > 5` is **valid**.

✗ COMMON MISTAKE

The GROUP BY rule: every non-aggregated column in `SELECT` must appear in `GROUP BY`.

Ignoring this is the #1 rookie error. (MySQL is loose about it — don't rely on that; standard SQL / PostgreSQL / SQL Server will throw.)

ADVANCED: GROUPING SETS, ROLLUP, CUBE (DATA-ENGINEERING FAVORITES)

- **ROLLUP(a, b)** — grand total + subtotals along a hierarchy: (a,b), (a), ().
- **CUBE(a, b)** — subtotals for **every combination**: (a,b), (a), (b), ().
- **GROUPING SETS((a),(b))** — pick exactly which groupings you want.

PATTERN RECOGNITION

"How many X per Y?" or "Total/Average X for each Y?" → `GROUP BY Y` + aggregate.

"Groups with more than N items" → `GROUP BY` + `HAVING COUNT(*) > N`.

"Subtotals + grand total in one query" → `ROLLUP` / `CUBE`.

INTERVIEW Q

"What's the difference between WHERE and HAVING?"

"WHERE filters rows before grouping and can't use aggregates. HAVING filters groups after GROUP BY and can use aggregates like `COUNT(*) > 5`. Both can appear in the same query — WHERE first for row filters, HAVING for group filters."

PRACTICE — GROUP BY & AGGREGATIONS

1. **Easy** LC 596. Classes More Than 5 Students
2. **Easy** LC 1050. Actors and Directors Who Cooperated At Least Three Times
3. **Easy** LC 1729. Find Followers Count
4. **Med** LC 1141. User Activity for the Past 30 Days I
5. **Med** LC 1211. Queries Quality and Percentage
6. **Med** LC 1193. Monthly Transactions I
7. **Med** LC 1174. Immediate Food Delivery II
8. **Med** LC 1084. Sales Analysis III

5. Joins — Every Type + Self Join

SYNTAX

```
SELECT a.col, b.col
FROM table_a a
INNER JOIN table_b b ON a.id = b.a_id;
```

Join	Returns	Use when
<code>INNER JOIN</code>	Only rows that match in both tables	You need common records only
<code>LEFT JOIN</code>	All rows from left + matches from right (NULL if no match)	Left table is the "spine"; keep all its rows
<code>RIGHT JOIN</code>	Mirror of LEFT (all from right)	Rare — just swap the tables and use LEFT
<code>FULL OUTER JOIN</code>	All rows from both, NULLs on the missing side	Comparing two sources, keeping everything
<code>CROSS JOIN</code>	Cartesian product — every row × every row	Building calendar/date grids; rare otherwise
<code>SELF JOIN</code>	Join a table to itself using two aliases	Hierarchies (employee-manager), comparing rows within a table

SELF JOIN EXAMPLE (EMPLOYEE EARNS MORE THAN MANAGER)

```
SELECT e.name AS employee
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

✗ COMMON MISTAKE

LEFT JOIN + WHERE on the right table = accidental INNER JOIN!

`LEFT JOIN orders o ON c.id = o.customer_id WHERE o.status = 'shipped'` — this drops customers with no order! Put the condition in the ON clause (`ON c.id = o.customer_id AND o.status = 'shipped'`) to preserve the LEFT JOIN behavior.

✗ COMMON MISTAKE

Ambiguous column names: If both tables have `id`, always prefix: `a.id`, `b.id`. Skipping this throws "column ambiguous" errors.

PATTERN RECOGNITION

"Include X even if no Y" / "list all X with their Y (if any)" → **LEFT JOIN**.

"Employees and their managers" / any row referencing another row in the same table → **SELF JOIN**.

"Find rows in A missing from B" → **LEFT JOIN B ON... WHERE b.id IS NULL** (anti-join, see next section).

INTERVIEW Q

"Difference between INNER JOIN and LEFT JOIN?"

"INNER JOIN returns only matching rows from both tables; LEFT JOIN returns **all** rows from the left plus matches from the right (NULL if no match). Use LEFT JOIN when the left table is your 'spine' and you need to keep rows even if the right side has nothing — e.g., listing all customers even those with zero orders."

PRACTICE — JOINS

1. **Easy** LC 175. Combine Two Tables
2. **Easy** LC 181. Employees Earning More Than Their Managers (self join)
3. **Easy** LC 197. Rising Temperature (self join with date logic)
4. **Easy** LC 577. Employee Bonus
5. **Med** LC 1158. Market Analysis I
6. **Med** LC 1341. Movie Rating
7. **Med** LC 570. Managers with at Least 5 Direct Reports
8. **Med** LC 1179. Reformat Department Table (join-adjacent — pivot)

 [SQL 50](#) → "Basic Joins" section

6. Anti-Join & Semi-Join Patterns

These aren't keywords — they're **patterns** using JOIN or EXISTS. Interviewers **love** testing these because most candidates panic.

ANTI-JOIN: "ROWS IN A THAT ARE NOT IN B"

3 equivalent ways (know all three, and their NULL gotcha):

```
-- Method 1: LEFT JOIN + IS NULL (most common)
SELECT a.*
FROM customers a
LEFT JOIN orders b ON a.id = b.customer_id
WHERE b.customer_id IS NULL;

-- Method 2: NOT EXISTS (★ NULL-safe, preferred by DBAs)
SELECT a.*
FROM customers a
WHERE NOT EXISTS (
    SELECT 1 FROM orders b WHERE b.customer_id = a.id
);

-- Method 3: NOT IN (⚠️ breaks on NULL – avoid)
SELECT * FROM customers
WHERE id NOT IN (SELECT customer_id FROM orders);
```

SEMI-JOIN: "ROWS IN A THAT HAVE AT LEAST ONE MATCH IN B" (BUT DON'T DUPLICATE A)

```
SELECT * FROM customers a
WHERE EXISTS (
    SELECT 1 FROM orders b WHERE b.customer_id = a.id
);
```

✗ COMMON MISTAKE

The classic **NOT IN** trap: If the subquery returns even one NULL, **NOT IN** returns **zero rows**. Always prefer **NOT EXISTS**, or add **WHERE col IS NOT NULL** inside the subquery.

🎯 PATTERN RECOGNITION

"Customers who never placed an order" / "products with no sales" / "users who didn't do X"
→ **Anti-join** pattern (LEFT JOIN + IS NULL, or NOT EXISTS).

"Customers who placed at least one order" (without duplicating them) → **Semi-join** (EXISTS).

PRACTICE — ANTI/SEMI JOINS

1. **Easy** LC 183. Customers Who Never Order
2. **Easy** LC 607. Sales Person
3. **Med** LC 1795. Rearrange Products Table (bonus: pivot)
4. **Med** LC 1965. Employees With Missing Information
5. **Med** LC 1517. Find Users With Valid E-Mails

7. Subqueries (Correlated vs Non-Correlated)

A **subquery** is a query inside another query. Three places it can go:

- **In WHERE / HAVING:** `WHERE col > (SELECT AVG(col) FROM t)`
- **In FROM (derived table):** `FROM (SELECT ... FROM t) AS sub`
- **In SELECT (scalar):** `SELECT (SELECT MAX(x) FROM t2) AS max_x, ...` (returns exactly one value)

NON-CORRELATED VS CORRELATED

	Non-Correlated	Correlated
Depends on outer query?	No — runs once , independently	Yes — references outer row; runs once per outer row
Speed	Fast	Slow (repeated execution)
Example	<code>WHERE salary > (SELECT AVG(salary) FROM emp)</code>	<code>WHERE salary > (SELECT AVG(salary) FROM emp e2 WHERE e2.dept = e1.dept)</code>

★ KEY INTERVIEW POINT

Correlated subquery = "for each row, check something." They're slow. Modern rewrite: use a **window function** instead. `salary > AVG(salary) OVER (PARTITION BY dept)` replaces the correlated form and runs faster.

✗ COMMON MISTAKE

Scalar subquery returning multiple rows = ERROR ("subquery returns more than one row").

Fix: add `LIMIT 1` or aggregate (MAX/MIN), or switch to IN/EXISTS.

🔗 PATTERN RECOGNITION

"Above average..." / "greater than the min/max of..." → non-correlated subquery in WHERE.

"Above their department's/category's average" → correlated subquery **or** (better) window function `AVG() OVER (PARTITION BY dept)`.

"Latest per group" → subquery with `MAX(date) GROUP BY ...` and join back, **or** ROW_NUMBER window function.

PRACTICE — SUBQUERIES

1. **Easy** LC 176. Second Highest Salary (subquery classic)
2. **Easy** LC 596. Classes More Than 5 Students
3. **Med** LC 1113. Reported Posts
4. **Med** LC 1204. Last Person to Fit in the Bus
5. **Med** LC 1978. Employees Whose Manager Left the Company
6. **Med** LC 626. Exchange Seats (self-referential logic)
7. **Med** LC 1164. Product Price at a Given Date
8. **Med** LC 1321. Restaurant Growth

8. CTEs — Common Table Expressions

A **CTE** is a **named, temporary result set** that lives only for the duration of one query. Written with `WITH`. It's the modern replacement for messy nested subqueries.

SYNTAX

```

WITH cte_name AS (
  SELECT col1, col2
  FROM some_table
  WHERE ...
)
SELECT * FROM cte_name WHERE ...;

-- Multiple CTEs chained (comma-separated, no extra WITH)
WITH
  cte1 AS (SELECT ...),
  cte2 AS (SELECT ... FROM cte1 ...)
SELECT * FROM cte2;

```

CTE VS SUBQUERY VS TEMP TABLE

	CTE	Subquery	Temp Table
Readability	✓ High (named, top-to-bottom)	✗ Nested, hard to read	✓ Good
Reusable in same query?	✓ Yes, multiple times	✗ Rewrite it each time	✓ Yes
Persists across queries?	✗ No — one query only	✗ No	✓ Whole session
Recursion?	✓ Recursive CTE	✗	✗

★ KEY INTERVIEW POINT

Use CTEs to solve any complex problem: break the problem into named steps ("clean_data", "ranked", "filtered"). This is the **#1 hack for hard interview problems** — it makes your logic visible and easy to reason about. Interviewers love it.

🎯 PATTERN RECOGNITION

"Complex multi-step logic" → CTEs, one per step.

"Need to reuse a subquery" → CTE (no need to rewrite).

"Filter by a window function" → CTE containing the window, then filter in outer query.

PRACTICE — CTEs

1. **Easy** LC 1731. The Number of Employees Which Report to Each Employee
2. **Med** LC 1934. Confirmation Rate
3. **Med** LC 1934. Confirmation Rate (multi-CTE approach)
4. **Med** LC 1174. Immediate Food Delivery II
5. **Med** LC 550. Game Play Analysis IV
6. **Med** LC 1907. Count Salary Categories
7. **Hard** LC 262. Trips and Users
8. **Hard** LC 185. Department Top Three Salaries

9. Recursive CTEs (Hierarchies)

A **Recursive CTE** is a CTE that **references itself**. It's how you traverse hierarchies (org charts, folder trees, categories with parents) and generate sequences (dates, numbers) in pure SQL.

SYNTAX (2 REQUIRED PARTS + UNION ALL)

```
WITH RECURSIVE cte_name AS (  
  -- 1. ANCHOR: the starting point (base case)  
  SELECT id, name, 1 AS level  
  FROM employees  
  WHERE manager_id IS NULL          -- CEO (root)  
  
  UNION ALL                        -- must be UNION ALL, not UNION  
  
  -- 2. RECURSIVE: reference the CTE itself  
  SELECT e.id, e.name, c.level + 1  
  FROM employees e  
  JOIN cte_name c ON e.manager_id = c.id  
)  
SELECT * FROM cte_name;
```

Note: SQL Server doesn't need the **RECURSIVE** keyword (just **WITH cte AS ...**). PostgreSQL, MySQL 8+, Oracle require **WITH RECURSIVE**.

GENERATE A DATE SERIES (COMMON PATTERN)

```
WITH RECURSIVE dates AS (  
  SELECT DATE('2026-01-01') AS d  
  UNION ALL  
  SELECT DATE_ADD(d, INTERVAL 1 DAY)  
  FROM dates  
  WHERE d < '2026-12-31'      -- terminating condition – MUST have one!  
)  
SELECT * FROM dates;
```

✗ COMMON MISTAKE

Forgetting the terminating condition = infinite loop! The recursive member MUST have a WHERE clause that eventually becomes false, otherwise the CTE runs forever (most engines throw a "max recursion depth" error).

✗ COMMON MISTAKE

Using `UNION` instead of `UNION ALL`: UNION removes duplicates each step — slower and can even break results. Always use `UNION ALL` in recursive CTEs.

🎯 PATTERN RECOGNITION

"Employee-manager hierarchy" / "all subordinates of X" / "folder tree" / "category with all children" → **Recursive CTE**.

"Fill in missing dates" / "generate a series of numbers" → Recursive CTE (or a calendar table).

"Bill of materials" / "graph traversal in SQL" → Recursive CTE.

INTERVIEW Q

"How would you find all direct and indirect reports of a manager?"

"That's a classic recursive CTE. Anchor row = the manager. Recursive step = join employees whose `manager_id` matches any id already in the CTE. UNION ALL them together, terminate when no more matches are found."

PRACTICE — RECURSIVE CTES (HARD, BUT HIGH-VALUE)

1. **Hard** LC 1384. Total Sales Amount by Year
2. **Hard** LC 1635. Hopper Company Queries I
3. **Hard** LC 1651. Hopper Company Queries III
4. **Hard** LC 1767. Find the Subtasks that Did Not Execute
5. **Hard** LC 1336. Number of Transactions per Visit
6. **Hard** LC 2118. Build the Equation
7. **Hard** LC 2308. Arrange Table by Gender

10. Window Functions ★★ ★ (The Most-Asked Topic)

Window functions are the **#1 differentiator** between beginner and hire-worthy SQL. They perform calculations across a set of rows related to the current row — **without collapsing them** (unlike GROUP BY, which does collapse).

💡 Analogy

GROUP BY = a blender: throws all rows into a jar and gives you back one summary row.

Window function = a photographer walking through a room: takes a snapshot of the group for each person, without merging anyone. You keep every row plus the group info.

THE UNIVERSAL SYNTAX

```
function_name(...) OVER (  
  [PARTITION BY col1, col2, ...]    -- split into groups (like GROUP BY)  
  [ORDER BY col1 [ASC|DESC], ...]   -- order within each group  
  [ROWS BETWEEN ... AND ...]       -- frame (optional, for running totals)  
)
```

THE 3 FLAVORS OF WINDOW FUNCTIONS

Category	Functions	Purpose
Ranking	<code>ROW_NUMBER()</code> , <code>RANK()</code> , <code>DENSE_RANK()</code> , <code>NTILE(n)</code> , <code>PERCENT_RANK()</code>	Rank rows within a partition
Value / Offset	<code>LAG()</code> , <code>LEAD()</code> , <code>FIRST_VALUE()</code> , <code>LAST_VALUE()</code> , <code>NTH_VALUE()</code>	Access neighboring rows
Aggregate as window	<code>SUM() OVER</code> , <code>AVG() OVER</code> , <code>COUNT()</code> <code>OVER</code> , <code>MAX() OVER</code> , <code>MIN() OVER</code>	Running totals, moving averages, group stats without collapsing

PARTITION BY VS GROUP BY (★ INTERVIEW CLASSIC)

GROUP BY

Collapses rows into **one row per group**.

You lose the individual rows.

Query returns fewer rows than the input.

PARTITION BY (window)

Keeps every row, but computes the value per partition.

Query returns the same number of rows as the input.

✗ COMMON MISTAKE

#1 window function mistake (VERY common in interviews):

`WHERE ROW_NUMBER() OVER (...) <= 3` — **SYNTAX ERROR** in every major SQL engine. Window functions are computed in the SELECT phase (step 6), but WHERE runs at step 3. The fix is **always** to wrap in a CTE or subquery, then filter in the outer query.

★ KEY INTERVIEW POINT

Window functions are calculated **after WHERE, GROUP BY, HAVING — but before DISTINCT and the final ORDER BY**. So the alias only exists in the outer query. This one fact explains 90% of window-function errors.

11. Ranking: ROW_NUMBER vs RANK vs

DENSE_RANK ★

Interviewers **love** asking which one to use — because they behave differently on ties, and the wrong choice gives wrong output. Memorize this table.

Salaries	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
100 (tie)	2	1	1
90	3	3 (skips 2)	2 (no skip)

- **ROW_NUMBER()** — every row gets a unique number, ties broken arbitrarily. Use when you need **exactly one** row per rank.
- **RANK()** — ties get the same rank, then **skips** the next numbers. Standard "olympic ranking."
- **DENSE_RANK()** — ties get the same rank, but the next rank is **not skipped**. Use for "top N distinct values per group."

★ KEY INTERVIEW POINT

Choosing between them for "Top N per group":

- "Top N employees each dept, exactly N rows even if ties" → **ROW_NUMBER**
- "Top N distinct salaries per dept (all tied employees included)" → **DENSE_RANK**
- "Standard leaderboard with ties" → **RANK**

CLASSIC PATTERN: 2ND/NTH HIGHEST SALARY (PER DEPARTMENT)

```
WITH ranked AS (  
    SELECT name, dept_id, salary,  
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk  
    FROM employees  
)  
SELECT name, dept_id, salary  
FROM ranked  
WHERE rnk = 2;    -- 2nd highest per department
```

PATTERN RECOGNITION

"Top N per group" / "Nth highest per something" / "top salary per department" →

ROW_NUMBER / RANK / DENSE_RANK with **PARTITION BY**.

"Assign each row a unique number in order" → **ROW_NUMBER**.

"Deduplicate keeping one row per key" → **ROW_NUMBER + WHERE rn = 1**.

PRACTICE — RANKING (★ HIGH-VALUE)

1. **Easy** LC 176. Second Highest Salary
2. **Med** LC 177. Nth Highest Salary
3. **Med** LC 178. Rank Scores
4. **Med** LC 184. Department Highest Salary
5. **Med** LC 1731. The Number of Employees Which Report to Each Employee
6. **Med** LC 1341. Movie Rating
7. **Med** LC 1149. Article Views II
8. **Hard** LC 185. Department Top Three Salaries (**DENSE_RANK** classic)
9. **Hard** LC 1454. Active Users
10. **Hard** LC 615. Average Salary: Departments VS Company

12. LEAD, LAG & Consecutive Rows

LAG(col, n) = value from **n rows BEFORE** the current row. **LEAD(col, n)** = value from **n rows AFTER**. Default n=1. Great for comparing "this row vs the previous/next row."

SYNTAX

```
LAG(col, offset, default) OVER (PARTITION BY ... ORDER BY ...)  
LEAD(col, offset, default) OVER (PARTITION BY ... ORDER BY ...)
```

MONTH-OVER-MONTH GROWTH

```
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
       revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month)) /
            LAG(revenue) OVER (ORDER BY month), 2) AS mom_pct
FROM monthly_sales;
```

PATTERN RECOGNITION

"Compare with previous/next row" → LAG / LEAD.

"Month-over-month" / "day-over-day change" → LAG.

"Time between events" / "delay between actions" → LAG on timestamp + subtract.

"Consecutive occurrences" (e.g., 3 days in a row) → LAG twice OR window with a range filter.

PRACTICE — LAG / LEAD

1. **Med** LC 180. Consecutive Numbers
2. **Med** LC 626. Exchange Seats
3. **Med** LC 1097. Game Play Analysis V
4. **Med** LC 1174. Immediate Food Delivery II
5. **Med** LC 550. Game Play Analysis IV (LAG-friendly)
6. **Hard** LC 601. Human Traffic of Stadium (consecutive days)
7. **Hard** LC 1225. Report Contiguous Dates

13. Running Totals & Moving Averages (Frames)

Any aggregate function becomes a **window aggregate** when you add `OVER`. Add a **frame clause** to define which rows are included in the calculation for each row.

FRAME SYNTAX

ROWS BETWEEN <start> **AND** <end>

-- Common frames:

UNBOUNDED PRECEDING -- from the very first row of the partition

n **PRECEDING** -- n rows before current

CURRENT ROW -- current row itself

n **FOLLOWING** -- n rows after current

UNBOUNDED FOLLOWING -- to the very last row

RUNNING TOTAL (CUMULATIVE SUM)

```
SELECT order_date, amount,
       SUM(amount) OVER (
         ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM orders;
```

7-DAY MOVING AVERAGE

```
SELECT dt, revenue,
       AVG(revenue) OVER (
         ORDER BY dt
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW    -- 7 rows total: today + 6 before
       ) AS avg_7d
FROM daily_sales;
```

✗ COMMON MISTAKE

ROWS vs RANGE for date-based windows: **ROWS** counts **physical rows**. If dates are missing (gaps), a 7-row window may not equal 7 calendar days. For true calendar-based windows, either densify dates first, or use **RANGE** (support varies by engine).

✗ COMMON MISTAKE

Forgetting **ORDER BY inside **OVER()****: for running totals, without **ORDER BY** the sum is over the whole partition — you get the grand total on every row instead of the cumulative. Always include **ORDER BY** for cumulative math.

PATTERN RECOGNITION

"Running total" / "cumulative sum" → **SUM() OVER (ORDER BY ... ROWS UNBOUNDED PRECEDING)**.

"7-day / 30-day moving average" → **AVG() OVER (ORDER BY dt ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)**.

"Percentage of total" per row → **col / SUM(col) OVER (PARTITION BY group) * 100**.

"Rolling max / min" → **MAX/MIN() OVER** with a frame.

PRACTICE — RUNNING & ROLLING

1. **Med** LC 534. Game Play Analysis III (running total)
2. **Med** LC 1321. Restaurant Growth (7-day rolling)
3. **Med** LC 579. Find Cumulative Salary of an Employee
4. **Med** LC 1613. Find the Missing IDs (window + gap detection)
5. **Hard** LC 618. Students Report by Geography
6. **Hard** LC 1225. Report Contiguous Dates
7. **Hard** LC 569. Median Employee Salary

14. Gaps & Islands (Hardest Pattern)

The **Gaps & Islands** problem = find **consecutive ranges** of data (islands) and the breaks between them (gaps). Comes up in "consecutive days," "login streaks," "user sessions," "consecutive numbers." **The hardest pattern DE candidates face.**

THE UNIVERSAL TRICK: ROW_NUMBER SUBTRACTION

If you have a sequence of dates/numbers, subtract `ROW_NUMBER()` from the value. Rows in the same "island" produce the same difference → group by that difference.

```
-- Find consecutive login date ranges per user
WITH numbered AS (
  SELECT user_id, login_date,
         DATE_SUB(
           login_date,
           INTERVAL ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) DAY
         ) AS grp
  FROM logins
)
SELECT user_id, MIN(login_date) AS streak_start,
       MAX(login_date) AS streak_end,
       COUNT(*) AS streak_length
FROM numbered
GROUP BY user_id, grp;
```

★ KEY INTERVIEW POINT

The intuition: in a consecutive sequence, both the value and the row number increase by 1 each step. So `value - row_number` stays constant for one island, and jumps when there's a gap. GROUP BY that difference → each group is one island. This is a killer trick that impresses interviewers.

🔗 PATTERN RECOGNITION

"Consecutive N days/months of X" / "streaks" / "unbroken run of..." → **Gaps & Islands** (ROW_NUMBER subtraction).

"Sessions of user activity" / "continuous periods" → same pattern with LAG to detect gap boundaries.

"Find missing IDs/dates in a sequence" → LEFT JOIN with a full number/date series, WHERE IS NULL.

PRACTICE — GAPS & ISLANDS (HARD, HIGH-IMPACT)

1. **Med** LC 180. Consecutive Numbers
2. **Hard** LC 601. Human Traffic of Stadium
3. **Hard** LC 1225. Report Contiguous Dates
4. **Hard** LC 1285. Find the Start and End Number of Continuous Ranges
5. **Hard** LC 1454. Active Users
6. **Hard** LC 2173. Longest Winning Streak
7. **Hard** LC 1972. First and Last Call On the Same Day

 **Advanced SQL 50** — many gaps & islands problems here

15. CASE WHEN & Conditional Logic

CASE WHEN is SQL's if/else — used for conditional columns, bucketing, and clever aggregations.

SYNTAX

```
-- Searched CASE (most flexible)
```

```
CASE
```

```
  WHEN condition1 THEN result1
```

```
  WHEN condition2 THEN result2
```

```
  ELSE default_result
```

```
END
```

```
-- Simple CASE (equality-only)
```

```
CASE col
```

```
  WHEN 'A' THEN 1
```

```
  WHEN 'B' THEN 2
```

```
  ELSE 0
```

```
END
```

KILLER TRICK: CONDITIONAL AGGREGATION (★)

Combine CASE inside SUM/COUNT/AVG to count/sum only rows matching a condition. This **replaces multiple queries with one** — a favourite interview pattern.

SELECT

```
COUNT(*) AS total_orders,  
SUM(CASE WHEN status = 'shipped' THEN 1 ELSE 0 END) AS shipped_count,  
SUM(CASE WHEN status = 'cancelled' THEN amount ELSE 0 END) AS lost_revenue  
FROM orders;
```

PATTERN RECOGNITION

"Bucket into categories" (Low/Med/High salary) → CASE WHEN in SELECT.

"Count of X where condition, in same query" → SUM(CASE WHEN condition THEN 1 ELSE 0 END).

"Pivot rows into columns" → CASE WHEN inside aggregation (see next section).

"If NULL then default" → `COALESCE` is cleaner than `CASE WHEN x IS NULL`.

PRACTICE — CASE WHEN

1. **Easy** LC 627. Swap Salary
2. **Med** LC 1873. Calculate Special Bonus
3. **Med** LC 1795. Rearrange Products Table
4. **Med** LC 1907. Count Salary Categories
5. **Med** LC 1179. Reformat Department Table (CASE + pivot)
6. **Med** LC 1965. Employees With Missing Information
7. **Hard** LC 262. Trips and Users

16. UNION, INTERSECT, EXCEPT (Set Operations)

Operator	What it does	Duplicates?
<code>UNION</code>	Combines rows from two queries	Removes duplicates (extra work)
<code>UNION ALL</code>	Combines rows, keeps duplicates	Faster — always prefer if dups don't matter
<code>INTERSECT</code>	Rows appearing in both queries	Removes duplicates
<code>EXCEPT</code> (or <code>MINUS</code> in Oracle)	Rows in query1 but NOT in query2	Removes duplicates

✗ COMMON MISTAKE

Rules that break UNION: both queries must have the **same number of columns**, in the **same order**, with **compatible data types**. Column names come from the first query.

✗ COMMON MISTAKE

Using UNION when UNION ALL would do: UNION does a de-duplication sort — slow on big tables. If you know results don't overlap, use UNION ALL. Interviewers ask why one is faster.

🎯 PATTERN RECOGNITION

"Combine two lists" (e.g., customers from two systems) → **UNION / UNION ALL**.

"Users who did both X and Y" → **INTERSECT** (or INNER JOIN).

"In A but not in B" → **EXCEPT** (or anti-join with NOT EXISTS).

17. Pivot & Unpivot (Rows ↔ Columns)

Pivot = turning row values into columns (e.g., monthly rows → 12 month columns). **Unpivot** = the reverse.

MANUAL PIVOT WITH CASE + GROUP BY (WORKS EVERYWHERE)

```
-- Convert row-based subject scores to column-based per student
SELECT student_id,
       SUM(CASE WHEN subject = 'Math' THEN score END) AS math,
       SUM(CASE WHEN subject = 'Science' THEN score END) AS science,
       SUM(CASE WHEN subject = 'English' THEN score END) AS english
FROM scores
GROUP BY student_id;
```

Some engines have a native `PIVOT` keyword (SQL Server, Oracle). The CASE + GROUP BY pattern above works **everywhere** — safer to know.

PATTERN RECOGNITION

"Show one row per user with a column per category/month/status" → **Pivot** with CASE + GROUP BY.

"Turn wide columns into long form" → **Unpivot** (UNION ALL each column).

PRACTICE — PIVOT

1. **Easy** LC 1179. Reformat Department Table
2. **Med** LC 1795. Rearrange Products Table
3. **Med** LC 618. Students Report by Geography
4. **Med** LC 1543. Fix Product Name Format

18. String Functions

Function	Purpose	Example
<code>LENGTH()</code> / <code>LEN()</code>	String length	<code>LENGTH('hello')</code> → 5
<code>UPPER()</code> , <code>LOWER()</code>	Change case	<code>UPPER('abc')</code> → ABC
<code>CONCAT()</code> / <code> </code>	Join strings	<code>CONCAT(first, ' ', last)</code>
<code>SUBSTRING(s, start, len)</code>	Extract portion	<code>SUBSTRING('hello', 2, 3)</code> → 'ell'
<code>TRIM()</code> , <code>LTRIM()</code> , <code>RTRIM()</code>	Remove spaces	<code>TRIM(' x ')</code> → 'x'
<code>REPLACE(s, find, repl)</code>	Replace all occurrences	<code>REPLACE('a-b', '-', '_')</code>
<code>LEFT(s, n)</code> , <code>RIGHT(s, n)</code>	First/last n chars	<code>LEFT('email@x.com', 5)</code>
<code>POSITION</code> / <code>INSTR</code> / <code>CHARINDEX</code>	Find position of substring	Varies by dialect
<code>REGEXP</code> / <code>RLIKE</code> / <code>~</code>	Regex matching	<code>WHERE email RLIKE '^[a-z]+@'</code>

✗ COMMON MISTAKE

Case sensitivity varies! MySQL is case-insensitive by default; PostgreSQL is case-sensitive. If in doubt, use `LOWER(col) = LOWER('value')` for a case-insensitive compare — but note this **breaks index usage** (see Optimization section).

19. Date/Time Functions ★ (DE-Critical)

Data engineering runs on dates — incremental loads, partitioning, time-series analysis. Get comfortable.

EXTRACTION

```

YEAR(dt), MONTH(dt), DAY(dt), HOUR(dt), MINUTE(dt)
EXTRACT(YEAR FROM dt)           -- ANSI standard
DATE_PART('month', dt)          -- PostgreSQL
DATEPART(month, dt)             -- SQL Server

```

ARITHMETIC (DIALECT DIFFERENCES!)

```

-- Add / subtract
DATEADD(day, 7, dt)             -- SQL Server
DATE_ADD(dt, INTERVAL 7 DAY)    -- MySQL
dt + INTERVAL '7 days'         -- PostgreSQL

-- Difference
DATEDIFF(day, d1, d2)           -- SQL Server (d2-d1)
DATEDIFF(d1, d2)                -- MySQL (d1-d2, days only)
d2 - d1                         -- PostgreSQL (returns interval)

```

TRUNCATION / FORMATTING

```

DATE_TRUNC('month', dt)         -- Postgres/Snowflake – start of month
DATE_FORMAT(dt, '%Y-%m')       -- MySQL – '2026-07'
TO_CHAR(dt, 'YYYY-MM')         -- Oracle/Postgres
FORMAT(dt, 'yyyy-MM')          -- SQL Server

```

✗ COMMON MISTAKE

Wrapping a date column in a function kills index use!

`WHERE YEAR(order_date) = 2026` — the engine can't use an index on `order_date`. Fix:

`WHERE order_date >= '2026-01-01' AND order_date < '2027-01-01'` — sargable, index-friendly.

Interviewers love this (it's called **sargability**).

✗ COMMON MISTAKE

Timezone gotcha: UTC vs local. Always store timestamps in **UTC** and convert on display.

Comparing "today" across servers in different regions is a classic bug.

PATTERN RECOGNITION

"Last N days" → `WHERE dt >= CURRENT_DATE - INTERVAL '7 days'`.

"Monthly / weekly aggregation" → `GROUP BY DATE_TRUNC('month', dt)`.

"Business days between" → DATEDIFF plus subtract weekends (harder — usually calendar table).

"Age of customer" → `DATEDIFF(year, dob, CURRENT_DATE)`.

PRACTICE — DATES

1. **Easy** LC 197. Rising Temperature
2. **Med** LC 1141. User Activity for the Past 30 Days I
3. **Med** LC 1193. Monthly Transactions I
4. **Med** LC 1729. Find Followers Count
5. **Med** LC 1204. Last Person to Fit in the Bus
6. **Med** LC 1321. Restaurant Growth

20. Finding & Removing Duplicates

FINDING DUPLICATES (WHICH KEYS APPEAR MORE THAN ONCE)

```
SELECT email, COUNT(*) AS cnt
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

GETTING THE ACTUAL DUPLICATE ROWS (NOT JUST KEYS)

```
WITH dupes AS (
  SELECT *,
    ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
  FROM users
)
SELECT * FROM dupes WHERE rn > 1;  -- the extras
```

DELETING DUPLICATES, KEEPING ONE

```
-- Approach: rank rows within duplicate groups, delete rn > 1
WITH ranked AS (
  SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
  FROM users
)
DELETE FROM users
WHERE id IN (SELECT id FROM ranked WHERE rn > 1);
```

PATTERN RECOGNITION

"Find duplicates" → GROUP BY key + HAVING COUNT(*) > 1.

"Delete duplicates, keep one" → ROW_NUMBER partitioned by dup key, delete rn > 1.

"Return only unique rows" → DISTINCT (simple) or ROW_NUMBER = 1 (when you need to pick a specific one).

PRACTICE — DUPLICATES

1. **Easy** LC 182. Duplicate Emails
2. **Easy** LC 196. Delete Duplicate Emails
3. **Med** LC 1050. Actors and Directors Who Cooperated At Least Three Times
4. **Med** LC 1204. Last Person to Fit in the Bus
5. **Med** LC 1174. Immediate Food Delivery II

21. Top-N per Group (★ Signature Pattern)

This pattern appears in almost **every** data engineering interview: "top 3 salaries per department," "latest 5 orders per customer," "most recent event per user." Recognize it instantly.

THE UNIVERSAL TEMPLATE

```

WITH ranked AS (
  SELECT t.*,
    ROW_NUMBER() OVER (
      PARTITION BY group_col      -- what defines "per group"
      ORDER BY sort_col DESC     -- what makes it "top" (DESC for max, ASC for min)
    ) AS rn
  FROM table t
)
SELECT * FROM ranked WHERE rn <= N;

```

★ KEY INTERVIEW POINT

Pick your ranking function based on ties:

- "Exactly 3 rows per group even with ties" → `ROW_NUMBER`
- "Top 3 distinct values, all tied employees included" → `DENSE_RANK`
- "Standard leaderboard with rank skipping" → `RANK`

🔗 PATTERN RECOGNITION

Trigger phrases: "**top N**", "**highest per**", "**latest per**", "**most recent per**", "**first N per**" → **Top-N per group** pattern.

PRACTICE — TOP-N PER GROUP

1. **Med** LC 184. Department Highest Salary (top 1)
2. **Hard** LC 185. Department Top Three Salaries (top 3 with ties)
3. **Med** LC 1341. Movie Rating
4. **Med** LC 1112. Highest Grade For Each Student
5. **Med** LC 1149. Article Views II
6. **Hard** LC 2004. The Number of Seniors and Juniors to Join the Company

22. DDL / DML / DCL / TCL (The Four SQL Families)

Family	Full name	Commands	Purpose
DDL	Data Definition Language	<code>CREATE</code> , <code>ALTER</code> , <code>DROP</code> , <code>TRUNCATE</code> , <code>RENAME</code>	Define/modify database structure
DML	Data Manipulation Language	<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , <code>MERGE</code>	Manipulate the data itself
DCL	Data Control Language	<code>GRANT</code> , <code>REVOKE</code>	Permissions/access control
TCL	Transaction Control Language	<code>BEGIN</code> , <code>COMMIT</code> , <code>ROLLBACK</code> , <code>SAVEPOINT</code>	Manage transactions
DQL	Data Query Language	<code>SELECT</code>	Read data (sometimes counted separately)

DELETE VS TRUNCATE VS DROP (★ CLASSIC INTERVIEW QUESTION)

	DELETE	TRUNCATE	DROP
Family	DML	DDL	DDL
What it removes	Selected rows	ALL rows	ENTIRE table (structure + data)
WHERE supported?	✅ Yes	❌ No	❌ No
Rollback possible?	✅ Yes (in transaction)	❌ No (in most DBs)	❌ No
Resets identity/auto-increment?	❌ No	✅ Yes	N/A
Speed	Slow (row by row)	Very fast	Fast
Fires triggers?	✅ Yes	❌ No	N/A

KEY CONSTRAINTS

- **PRIMARY KEY** — uniquely identifies each row; **NOT NULL + UNIQUE**. One per table.
- **FOREIGN KEY** — references a PK in another table; enforces referential integrity.
- **UNIQUE** — no duplicates in this column; unlike PK, allows one NULL (most DBs).
- **NOT NULL** — value must be present.
- **CHECK** — custom validation rule (e.g., `CHECK (age >= 18)`).
- **DEFAULT** — auto-fill value if none provided.

23. Indexes (Clustered vs Non-Clustered)

An **index** is a lookup structure (usually a B-tree) that lets the engine find rows without scanning the whole table. Think of it as the index at the back of a book.

	Clustered Index	Non-Clustered Index
What it does	Determines the physical order of rows on disk	Separate structure with pointers to the data rows
How many per table?	Only 1 (data can be sorted one way)	Many (usually up to ~999)
Default	Automatically created on the PRIMARY KEY in SQL Server	Created explicitly with CREATE INDEX
Speed	Fastest for range scans	Slightly slower (extra lookup)

COMPOSITE INDEXES & THE "LEFTMOST PREFIX" RULE

An index on `(a, b, c)` can serve queries filtering on `a`, `(a,b)`, or `(a,b,c)` — but NOT on just `b` or `c`. Order the columns from most-filtered to least-filtered.

✗ COMMON MISTAKE

Sargability — the #1 index-killer: wrapping a column in a function in WHERE stops the engine from using its index. `WHERE YEAR(dt) = 2026`, `WHERE LOWER(name) = 'ruchika'`, `WHERE col + 1 = 5` — all break index usage. Fix: keep the column bare, transform the constant instead.

WHEN INDEXES HURT

- Every **INSERT/UPDATE/DELETE** must update every index → too many indexes slow writes.
- Indexes take **disk space**.
- On tiny tables, a full scan may be faster than an index lookup.
- Low-selectivity columns (e.g., a "gender" column with 2 values) are usually bad index candidates.

INTERVIEW Q

"Difference between clustered and non-clustered indexes?"

"A clustered index defines the physical order of rows on disk, so you can only have one per table. A non-clustered index is a separate structure holding pointers to the data rows, so you can have many. Clustered is fastest for range scans; non-clustered needs one extra lookup but supports multiple query patterns."

24. Views & Materialized Views

VIEW (VIRTUAL TABLE)

```
CREATE VIEW active_customers AS
SELECT id, name, email FROM customers WHERE status = 'active';

-- Use it like a table
SELECT * FROM active_customers WHERE ...;
```

A **view** is a **saved query** — it stores no data itself. Every time you query it, the underlying SELECT runs. Uses: simplify complex joins for users, provide a security layer (limit visible columns), abstract schema changes.

MATERIALIZED VIEW

Same as a view, but the **results are physically stored**. Great for expensive analytical queries — huge speedup. Downside: data goes stale; needs a **refresh strategy** (manual, scheduled, or on-commit depending on the DB).

	View	Materialized View
Stores data?	No (just the query)	Yes (physical copy)
Speed	Same as the underlying query	Fast — reads a stored table
Freshness	Always fresh	Can be stale; needs refresh
Use for	Simplification, security	Expensive analytics, dashboards

25. Stored Procedures & Functions

STORED PROCEDURE

```
CREATE PROCEDURE GetEmployeesByDept(IN dept_id INT)
BEGIN
  SELECT * FROM employees WHERE department_id = dept_id;
END;

-- Call it
CALL GetEmployeesByDept(5);
```

STORED PROCEDURE VS FUNCTION

	Stored Procedure	Function
Returns	Optional (0, 1, or many result sets)	Always returns a value
Called by	<code>CALL</code> / <code>EXEC</code>	Used inside a query, like a built-in
Can modify data (DML)?	✓ Yes	Usually not (varies by DB)
Can use IN/OUT parameters?	✓ Both	Only IN
Can use transactions inside?	✓ Yes	Usually no

26. Triggers

A **trigger** is code that automatically runs in response to an event on a table — **BEFORE** / **AFTER** an **INSERT**, **UPDATE**, or **DELETE**. Use for auditing, enforcing rules, or auto-updating derived columns.

```
CREATE TRIGGER log_salary_change
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
  INSERT INTO salary_audit(emp_id, old_sal, new_sal, changed_at)
  VALUES (OLD.id, OLD.salary, NEW.salary, NOW());
END;
```

Best practice: triggers hide business logic — they can be surprising and hard to debug. Use them sparingly; often application code or a stored procedure is clearer.

27. ACID, Transactions & Isolation Levels

A **transaction** is a group of SQL statements treated as a single unit — either all succeed (COMMIT) or all get undone (ROLLBACK).

ACID (MEMORIZE THIS)

Letter	Property	Meaning
A	Atomicity	All-or-nothing: transaction fully succeeds or fully rolls back
C	Consistency	DB moves from one valid state to another; constraints hold
I	Isolation	Concurrent transactions don't interfere with each other
D	Durability	Once committed, data survives crashes (persisted to disk)

ISOLATION LEVELS (FROM LOOSEST TO STRICTEST)

Level	Dirty reads	Non-repeatable reads	Phantom reads
Read Uncommitted	Possible	Possible	Possible
Read Committed (default in Postgres/SQL Server)	Prevented	Possible	Possible
Repeatable Read (default in MySQL)	Prevented	Prevented	Possible
Serializable	Prevented	Prevented	Prevented

Dirty read = reading another transaction's uncommitted change. **Non-repeatable read** = same row's value changes if you re-read it. **Phantom read** = new rows appear when you re-run the same query. Stricter = safer but slower.

28. Normalization (1NF, 2NF, 3NF, BCNF)

Normalization = organizing tables to reduce redundancy and dependency issues.

- **1NF (First Normal Form)**: each column holds an atomic value (no lists in a cell). Each row is unique.
- **2NF**: 1NF + every non-key column depends on the **whole** primary key (matters for composite PKs).
- **3NF**: 2NF + no **transitive dependencies** (non-key columns don't depend on other non-key columns). Move them to their own table.
- **BCNF**: stricter 3NF — every determinant is a candidate key.

Denormalization (deliberately duplicating data) is common in analytics/warehouses — it trades storage for query speed.

29. Star vs Snowflake Schema (Data Engineering)

★ Star Schema

Central **fact table** (transactions, events) connected directly to **dimension tables** (customer, product, date).

✓ **Simple, fast queries, easy for BI tools**

✗ **Some redundancy in dimensions**

❄ Snowflake Schema

Dimensions are **further normalized** into sub-dimensions (e.g., Product → Category → Department).

✓ **Less redundancy**

✗ **More joins, slower queries**

Fact tables hold numeric measurements (sales_amount, quantity). **Dimension tables** hold descriptive attributes (customer_name, product_name, date info). This is the core of warehouse modeling. **For DE interviews:** star schema is preferred for most BI use cases.

30. Query Optimization & EXPLAIN

Data engineers must write queries that scale to millions/billions of rows. Know these levers cold.

THE OPTIMIZATION CHECKLIST

1. **Read the execution plan** (`EXPLAIN` in Postgres/MySQL, `EXPLAIN ANALYZE` for actual runtime, "Include Actual Execution Plan" in SQL Server). Look for full table scans on big tables — that's your first target.
2. **Add appropriate indexes** on columns used in WHERE, JOIN, and ORDER BY. But not too many (slows writes).
3. **Keep queries sargable** — don't wrap indexed columns in functions in WHERE.
4. **Filter early** — reduce rows before joining. Put restrictive WHERE conditions on the largest tables.
5. **Avoid** `SELECT *` in production — read only the columns you need (helps columnar/covering indexes).
6. **Use** `EXISTS` **instead of** `IN` for large subqueries (and it's NULL-safe).

7. **Use** `UNION ALL` instead of `UNION` when duplicates don't matter (skips a sort).
8. **Partition large tables** by a filter column (date, region) → the engine can **prune partitions** and skip most of the data.
9. **Use appropriate data types** — INT beats VARCHAR for numeric keys; DATE beats VARCHAR for dates.
10. **Update statistics** so the optimizer picks good plans.

★ KEY INTERVIEW POINT

Sargability (SARG = Search ARGument ABLE) — a WHERE condition is sargable if the engine can use an index for it. Wrapping a column in a function (`YEAR(dt)` , `LOWER(name)`) makes it **non-sargable**. Rewrite to keep the column bare and transform the constant. This is **the #1 optimization interview question**.

PARTITION PRUNING (★ DE-SPECIFIC)

If a table is partitioned by `order_date` , filtering `WHERE order_date = '2026-03-27'` makes the engine read **only that one partition**. Turns a minutes-long query into a seconds-long one. But **only works if the column is bare** — `WHERE YEAR(order_date) = 2026` may skip the pruning.

INTERVIEW Q

"Your query is slow on a big table. How do you optimize?"

Structured answer: "First I'd run **EXPLAIN** to find the bottleneck — usually a full table scan or missing index. Then I'd check for **sargability** issues (functions on WHERE columns), add or use the right **index**, ensure **partition pruning** works, **filter early** to shrink the working set, avoid `SELECT *` , and prefer `EXISTS` over `IN` for large subqueries. On large joins, check if a smaller table can be broadcast/hashed."

PRACTICE — OPTIMIZATION & DE-HEAVY

1. Read one `EXPLAIN` plan on any of your existing queries — practice reading it.
2. Design a star schema for an e-commerce dataset (fact_orders + dim_customer/dim_product/dim_date).
3. Read: use-the-index-luke.com (best free indexing resource).



Complete Pattern Recognition Cheat Sheet

This is **the most important table in the doc**. Read the question → find the trigger phrase → know the pattern instantly. Master this and you'll recognize 90% of SQL interview problems on sight.

Trigger phrase in the question	Pattern / SQL construct to reach for
"How many X per Y?" / "Total X for each Y" / "Average X by Y"	GROUP BY + aggregate
"With more than N of..." / "at least N times"	GROUP BY + HAVING
"Top N" / "N-th highest" / "highest per group" / "latest per user"	Window function with ROW_NUMBER / RANK / DENSE_RANK — Top-N-per-group pattern
"Second highest" / "Nth highest" (with ties)	DENSE_RANK() OVER (ORDER BY ... DESC) → filter = N
"Running total" / "cumulative sum" / "so far"	SUM() OVER (ORDER BY ... ROWS UNBOUNDED PRECEDING)
"7-day / 30-day rolling avg" / "moving average"	AVG() OVER (ORDER BY dt ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
"Compare with previous / next row" / "MoM growth"	LAG / LEAD
"Consecutive days/months/values" / "streaks" / "unbroken run"	Gaps & Islands (ROW_NUMBER subtraction trick)
"Find missing dates/IDs in a sequence"	Generate full series (recursive CTE / calendar table) + LEFT JOIN + IS NULL
"Never did X" / "no matching Y" / "without any Z"	Anti-join: LEFT JOIN + IS NULL, or NOT EXISTS
"At least one match" without duplicating	Semi-join: EXISTS
"Employee-manager" / "hierarchy" / "all descendants of..."	Recursive CTE
"Employees earning more than their manager" (same table, two references)	Self join

"Above/below their group's average"	Correlated subquery OR window: <code>AVG() OVER (PARTITION BY group)</code>
"Above the overall average"	Non-correlated subquery: <code>WHERE x > (SELECT AVG(x) FROM t)</code>
"Duplicate rows" / "find dups"	GROUP BY key + HAVING COUNT(*) > 1
"Remove duplicates, keep one"	ROW_NUMBER partitioned by dup key → delete where rn > 1
"Bucket into Low / Med / High"	CASE WHEN in SELECT
"Count of X where condition — in same query"	SUM(CASE WHEN cond THEN 1 ELSE 0 END) — conditional aggregation
"Rows to columns" / "one row per user with a column per month"	Pivot: CASE + GROUP BY
"Combine two lists"	UNION (dedup) or UNION ALL (faster, keeps dups)
"In both A and B"	INTERSECT or INNER JOIN
"In A but not in B"	EXCEPT / MINUS, or anti-join
"Complex multi-step logic"	Chain multiple CTEs , one per step
"Filter using a window function result"	Put window in a CTE , then filter in outer query
"Last N days" / "past week"	<code>WHERE dt >= CURRENT_DATE - INTERVAL '7 days'</code>
"Monthly aggregate"	<code>GROUP BY DATE_TRUNC('month', dt)</code> — never wrap the column in a function in WHERE
"NULL/no match should show as 0"	LEFT JOIN + COALESCE (col, 0)
"Median" / "percentiles"	<code>PERCENTILE_CONT</code> / <code>NTILE(4)</code> for quartiles
"Rank into equal-size buckets"	<code>NTILE(n)</code>
"First / last value in a group"	<code>FIRST_VALUE()</code> / <code>LAST_VALUE()</code> , or ROW_NUMBER=1

Query is slow → what to check

EXPLAIN plan → check sargability, indexes, filter early, partition pruning

How to use this in a real interview: When you get a question, **say the pattern out loud before writing code**: "This is a top-N-per-group problem, so I'll use `ROW_NUMBER()` with `PARTITION BY`." That structured approach is what interviewers score highest.



Syntax Quick-Reference (What You Can/Can't Do)

The most-confused syntax rules in one place. Skim this before every interview.

Clause / feature	✅ You CAN	❌ You CANNOT
WHERE	Filter rows before grouping; use column values, subqueries	Use aggregate functions (COUNT/SUM); use SELECT aliases; use window function aliases
HAVING	Filter groups after GROUP BY; use aggregates	Reference row-level columns not in GROUP BY (only grouped cols + aggregates)
SELECT with GROUP BY	Include aggregates + columns in GROUP BY	Include a bare column NOT in GROUP BY (in standard SQL)
ORDER BY	Use column names, SELECT aliases, column positions (1, 2, 3)	Reliably rely on it in a subquery (ordering isn't guaranteed in nested selects)
Window function	Use in SELECT and ORDER BY	Use in WHERE, GROUP BY, HAVING (wrap in CTE)
NULL	Test with IS NULL / IS NOT NULL; use COALESCE	Compare with <code>=</code> or <code><></code> (<code>x = NULL</code> is never true)
NOT IN	Use with sets guaranteed non-NULL	Use with subqueries that may return NULL (returns 0 rows — use NOT EXISTS)
Recursive CTE	Have anchor + recursive members joined by UNION ALL , with a terminating condition	Use UNION (breaks recursion); omit the base case; forget the terminator (infinite loop)
LEFT JOIN condition	Put right-table filters in the ON clause to preserve left rows	Put right-table filters in WHERE (turns it into an INNER JOIN accidentally)
UNION	Combine queries with same column count & compatible types	Combine mismatched column counts or types
DISTINCT	Remove exact-duplicate rows across ALL selected columns	Deduplicate on some columns while keeping others — use ROW_NUMBER instead

Index-friendly WHERE	Keep the column bare: <code>WHERE dt >= '2026-01-01'</code>	Wrap the column in a function: <code>WHERE YEAR(dt) = 2026</code> (kills the index)
Column alias	Use in ORDER BY, HAVING (in most DBs)	Use in WHERE, GROUP BY (repeat the expression instead)

50+ Most-Asked Interview Questions (with Crisp Answers)

Skim this the night before. Every one of these is a real, frequently-asked question.

Foundations

1. Difference between DELETE, TRUNCATE, and DROP?

DELETE removes selected rows (DML, has WHERE, rollback-able, slow). TRUNCATE removes all rows (DDL, no WHERE, fast, resets identity). DROP removes the entire table structure + data. TRUNCATE and DROP can't be rolled back in most DBs.

2. Difference between WHERE and HAVING?

WHERE filters rows before GROUP BY and can't use aggregates. HAVING filters groups after GROUP BY and can use aggregates like `COUNT(*) > 5`.

3. Explain SQL execution order.

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.
This explains why you can't use SELECT aliases in WHERE.

4. What is a primary key vs a unique key?

PRIMARY KEY = NOT NULL + UNIQUE, one per table. UNIQUE = no duplicates but allows one NULL, many per table.

5. Difference between UNION and UNION ALL?

UNION removes duplicates (does a sort/dedup — slower). UNION ALL keeps all rows including duplicates (faster). Prefer UNION ALL when you know there's no overlap.

6. COUNT(*) vs COUNT(col) vs COUNT(DISTINCT col)?

`COUNT(*)` counts all rows. `COUNT(col)` counts non-NULL values in that column.

`COUNT(DISTINCT col)` counts unique non-NULL values.

7. How do you handle NULLs correctly?

Never use `=` or `<>` with NULL — use `IS NULL` / `IS NOT NULL`. Replace with `COALESCE(col, default)`. Aggregates ignore NULLs (except `COUNT(*)`). `NOT IN` with NULL returns nothing — prefer `NOT EXISTS`.

8. Difference between CHAR and VARCHAR?

CHAR is fixed-length (padded with spaces). VARCHAR is variable-length. Use VARCHAR for most text; CHAR only for truly fixed-width values (like country codes).

Joins & Set Operations

9. Difference between INNER JOIN and LEFT JOIN?

INNER returns only matched rows from both. LEFT returns all rows from the left plus matches from the right (NULL where no match). Use LEFT when the left table is your spine.

10. What's a self join? Give an example.

Joining a table to itself using two aliases. Classic: "employees who earn more than their manager" — join `employees e` with `employees m` on `e.manager_id = m.id`.

11. Find customers with no orders.

Anti-join: `LEFT JOIN orders ON ... WHERE orders.id IS NULL`, or `WHERE NOT EXISTS (SELECT 1 FROM orders WHERE ...)`.

12. Difference between EXCEPT/MINUS and NOT IN?

EXCEPT/MINUS returns rows in A not in B (whole-row comparison, dedup'd). NOT IN checks a single column and breaks on NULL. NOT EXISTS is usually the safest.

13. What happens if you put a LEFT JOIN filter in WHERE vs ON?

Filter in ON keeps unmatched left rows (with NULLs). Filter in WHERE excludes them — silently turning your LEFT JOIN into an INNER JOIN. Classic bug.

Grouping & Aggregation

14. Find departments with more than 5 employees.

```
SELECT dept, COUNT(*) FROM emp GROUP BY dept HAVING COUNT(*) > 5 .
```

15. What are ROLLUP and CUBE?

ROLLUP produces subtotals along a hierarchy plus a grand total. CUBE produces subtotals for every combination of the grouping columns. Both extend GROUP BY.

16. Count shipped, cancelled, and pending orders in a single query.

Conditional aggregation: `SUM(CASE WHEN status='shipped' THEN 1 ELSE 0 END) AS shipped,`
...

Subqueries & CTEs

17. Difference between correlated and non-correlated subquery?

Non-correlated runs once, independent of the outer query. Correlated references outer columns and runs once per outer row — slow. Often replaceable with a window function.

18. What is a CTE and when do you use it?

A named temporary result set defined with `WITH`. Use for readability (break complex logic into named steps), for reuse in the same query, and for recursion.

19. CTE vs subquery vs temp table?

CTE = one query, named, readable, recursive. Subquery = inline, harder to reuse. Temp table = persists through the whole session, can be indexed, best for very heavy multi-query pipelines.

20. Write a recursive CTE for an employee-manager hierarchy.

```
WITH RECURSIVE h AS (SELECT id, name, 1 AS lvl FROM emp WHERE manager_id IS NULL UNION ALL SELECT e.id, e.name, h.lvl+1 FROM emp e JOIN h ON e.manager_id = h.id) SELECT * FROM h;
```

Window Functions

21. Difference between ROW_NUMBER, RANK, and DENSE_RANK?

All rank rows. ROW_NUMBER = unique numbers, breaks ties arbitrarily. RANK = same rank for ties, then **skips**. DENSE_RANK = same rank for ties, next rank **not skipped**.

22. Find the 2nd highest salary in each department.

```
WITH r AS (SELECT *, DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk FROM emp) SELECT * FROM r WHERE rnk = 2;
```

23. Difference between PARTITION BY and GROUP BY?

GROUP BY collapses rows into one per group. PARTITION BY (in a window function) keeps all rows and computes per group — you don't lose row-level detail.

24. Calculate a 7-day rolling average of daily revenue.

```
AVG(revenue) OVER (ORDER BY dt ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
```

25. Calculate month-over-month growth.

Use LAG: `revenue - LAG(revenue) OVER (ORDER BY month)` for change, divide for %.

26. Why can't you use a window function directly in WHERE?

Window functions execute in the SELECT phase, but WHERE runs earlier. The alias doesn't exist yet. Wrap the window in a CTE, filter in the outer query.

27. Find the top 3 salaries per department (include ties).

Use DENSE_RANK, not ROW_NUMBER, so tied employees are all included. Filter `WHERE rnk <= 3`.

28. Find consecutive days a user logged in.

Gaps & Islands: `DATE_SUB(login_date, INTERVAL ROW_NUMBER() OVER (PARTITION BY user ORDER BY login_date) DAY)` stays constant for one streak. Group by that.

Duplicates & Deduplication

29. How do you find duplicate emails?

```
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
```

30. How do you delete duplicates keeping one row per email?

Add `ROW_NUMBER() OVER (PARTITION BY email ORDER BY id)` in a CTE, delete rows where rn > 1.

Indexes & Optimization

31. Clustered vs non-clustered index?

Clustered defines the physical row order — one per table. Non-clustered is a separate structure with pointers — many per table.

32. What is sargability?

Whether a query can use an index. Wrapping a column in a function in WHERE (`YEAR(dt)`, `LOWER(name)`) breaks sargability. Keep the column bare, transform the constant instead.

33. When does an index hurt performance?

On heavy INSERT/UPDATE/DELETE workloads (every index must be updated), on tiny tables, and on low-selectivity columns (like a "gender" flag).

34. How do you optimize a slow query?

Read `EXPLAIN`, look for full scans; ensure indexes on WHERE/JOIN columns; make WHERE sargable; filter early; avoid `SELECT *`; use EXISTS over IN; leverage partition pruning; check statistics.

35. What is partition pruning?

When a partitioned table's query filters on the partition column, the engine reads only matching partitions and skips the rest — massively faster.

Views, SPs, Triggers

36. View vs materialized view?

A view is a saved query — no data stored, always fresh. A materialized view physically stores the result — fast but needs refresh strategy.

37. Stored procedure vs function?

Procedures may return 0/many result sets, are called with `CALL`, can modify data and use transactions. Functions always return a value and are used inside queries.

38. What's a trigger and when would you use one?

Code that fires automatically before/after INSERT/UPDATE/DELETE on a table. Use for auditing, enforcing rules, or auto-updating derived fields. Use sparingly — they hide logic.

Transactions & Concurrency

39. Explain ACID.

Atomicity (all-or-nothing), Consistency (valid state to valid state), Isolation (concurrent txns don't interfere), Durability (committed data survives crashes).

40. What are isolation levels?

Read Uncommitted (dirty reads possible), Read Committed (default in Postgres/SQL Server), Repeatable Read (default in MySQL), Serializable (strictest). Trade off between concurrency and safety.

41. What is a deadlock?

Two transactions each holding a lock the other needs, so both wait forever. The engine detects it and rolls back one. Avoid with consistent lock ordering and short transactions.

Data Modeling

42. What is normalization? Explain 1NF, 2NF, 3NF.

Reducing redundancy. 1NF = atomic values, unique rows. 2NF = 1NF + full functional dependency on the whole PK. 3NF = 2NF + no transitive dependencies (non-key column depending on another non-key column).

43. When would you denormalize?

In analytics/warehouses where query speed matters more than storage — pre-joining dimensions into facts, storing computed columns. Common in star schemas.

44. Star vs snowflake schema?

Star = fact table joined directly to dimensions (simple, fast, some redundancy). Snowflake = dimensions further normalized into sub-dimensions (less redundancy, more joins, slower). Star is preferred for BI.

45. What are fact vs dimension tables?

Fact tables hold numeric measurements (sales_amount, quantity) and foreign keys. Dimension tables hold descriptive attributes (customer name, product name, date attributes).

DE-Specific & Scenarios

46. How do you implement incremental loading in SQL?

Track the last processed value (watermark: timestamp or ID). Read only rows where `modified_at > @last_watermark`. Apply via UPSERT/MERGE. Update the watermark on success.

47. Explain Slowly Changing Dimensions (SCD Type 1 vs 2).

Type 1 overwrites the old value (no history). Type 2 inserts a new versioned row with effective_from/effective_to dates, keeping full history.

48. Latest record per user — how do you write it?

`ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_ts DESC) = 1`. Or a self-join with `MAX(event_ts) GROUP BY user_id`.

49. Find users active on N consecutive days.

Gaps & Islands: subtract ROW_NUMBER from the date, group by (user, difference), keep groups with count $\geq N$.

50. Calculate D1 / D7 retention.

For each signup cohort, count users who came back exactly 1 (or 7) days later divided by total in the cohort. LEFT JOIN signups with events on `event_date = signup_date + INTERVAL '1 day'`, group by signup_date.

51. Find employees earning above their department's average.

Window function: `WHERE salary > AVG(salary) OVER (PARTITION BY dept)` (wrap in CTE — can't do this directly in WHERE).

52. Pivot monthly sales into 12 month columns per year.

Manual pivot: `SUM(CASE WHEN MONTH(dt)=1 THEN amount END) AS jan, ...` GROUP BY year.

53. How would you find the median salary?

`PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary)` (Postgres/SQL Server). Or workaround with ROW_NUMBER + COUNT.

54. What is a covering index?

An index that contains all columns needed by a query, so the engine reads only the index without touching the base table — very fast. Called "index-only scan."

55. How would you design tables for a many-to-many relationship?

Use a junction/bridge table with foreign keys to both entities (e.g., `student_course(student_id, course_id)`), with a composite primary key.

Final advice for interviews: for each problem — (1) restate the question and clarify edge cases (NULLs, ties, empty results), (2) say the pattern out loud ("This is a top-N-per-group problem"), (3) write step-by-step using CTEs (don't try to nail it in one shot), (4) mention performance considerations at the end. Do this and you'll stand out. All the best Ruchika! 🚀